Practical 17

17	Create department table with the following structure Field Name: Deptno, DeptName, Location Data Type: Int, Varchar(30), Varchar(30) a) Add column designation to the department table. b) Insert values into the table. c) List the records of dept table grouped by deptno. d) Update the record where deptno is 9. e) Delete any column data from the table.	2

```
CREATE TABLE Department (
   Deptno INT PRIMARY KEY,
   DeptName VARCHAR(30),
   Location VARCHAR(30)
);
ALTER TABLE Department
ADD Designation VARCHAR(30);
INSERT INTO Department VALUES (1, 'HR', 'New York', 'Manager');
INSERT INTO Department VALUES (2, 'IT', 'London', 'Developer');
INSERT INTO Department VALUES (3, 'Finance', 'Delhi', 'Analyst');
SELECT Deptno, COUNT(*)
FROM Department
GROUP BY Deptno;
UPDATE Department
SET DeptName = 'Admin', Location = 'Mumbai'
WHERE Deptno = 9;
UPDATE Department
SET Designation = NULL;
```

1. Add column designation
2. Insert values
3. List records grouped by deptno
4. Update record where deptno is 9
5. Delete any column data (example: Designation data)

Output:

Results Explain Describe Saved SQL History

DEPTNO	DEPTNAME	LOCATION	DESIGNATION
1	HR	New York	-
2	IT	London	-
3	Finance	Delhi	-

3 rows returned in 0.00 seconds CSV Export

Practical 18

18	Create department table with the following structure Field Name: Deptno — Data Type: Int Field Name: DeptName — Data Type: Varchar(30) Field Name: Location — Data Type: Varchar(30) a) Add column designation to the department table. b) Insert values into the table. c) List the records of dept table grouped by deptno. d) Update the record where deptno is 9. e) Delete any column data from the table.

```
CREATE TABLE Department2 (
   Deptno INT PRIMARY KEY,
   DeptName VARCHAR(30),
   Location VARCHAR(30)
);
ALTER TABLE Department2
ADD Designation VARCHAR(30);
INSERT INTO Department2 VALUES (10, 'Sales', 'Paris', 'Executive');
INSERT INTO Department2 VALUES (11, 'Support', 'Berlin', 'Assistant');
SELECT Deptno, COUNT(*)
FROM Department2
GROUP BY Deptno;
UPDATE Department2
SET DeptName = 'Operations'
WHERE Deptno = 9;
UPDATE Department2
SET Designation = NULL;
Select * from Department2;
```

Output:

Results Explain Describe Saved SQL History

DEPTNO	DEPTNAME	LOCATION	DESIGNATION
10	Sales	Paris	-
11	Support	Berlin	-

2 rows returned in 0.02 seconds CSV Export

Q19 Write PL/SQL code to display employee details using Explicit Cursors ?

```
CREATE TABLE EMPLOYEE2(empId int ,empName varchar(40),designation varchar(30));


INSERT INTO EMPLOYEE2 values (101,'xyz','ANDROID DEVLOPER');
INSERT INTO EMPLOYEE2 values (102,'wyz','Frontend DEVLOPER');
INSERT INTO EMPLOYEE2 values (103,'SHAM','DEVOPS DEVLOPER');
INSERT INTO EMPLOYEE2 values (104,'RAJ','ML DEVLOPER');
INSERT INTO EMPLOYEE2 values (105,'RAM','ANDROID DEVLOPER');


DECLARE
  CURSOR emp IS SELECT empId ,empName,designation  FROM EMPLOYEE2;
  v_empid EMPLOYEE.empId%TYPE;
  v_empName EMPLOYEE.empName%TYPE;
  v_designation EMPLOYEE.designation%TYPE;




BEGIN
OPEN emp;
LOOP
  FETCH emp INTO v_empid,v_empname,v_designation;
      EXIT WHEN emp%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(v_empid || ' ' || v_empname || '  ' || v_designation);
 END LOOP;
CLOSE emp;

END;
```

OUTPUT:

END;

Results Explain Describe Saved SQL History

```
105 RAM  ANDROID DEVLOPER
104 RAJ  ML DEVLOPER
103 SHAM  DEVOPS DEVLOPER
102 wyz  Frontend DEVLOPER
101 xyz  ANDROID DEVLOPER

Statement processed.
```

0.00 seconds

Q20. PL/SQL code to retrieve employee name, join date, and designation from employee database of an employee whose number is input by the user.

```
CREATE TABLE EMPLOYEE2(empId int ,empName varchar(40),designation varchar(30));

INSERT INTO EMPLOYEE2 values (101,'xyz','ANDROID DEVLOPER');
INSERT INTO EMPLOYEE2 values (102,'wyz','Frontend DEVLOPER');
INSERT INTO EMPLOYEE2 values (103,'SHAM','DEVOPS DEVLOPER');
INSERT INTO EMPLOYEE2 values (104,'RAJ','ML DEVLOPER');
INSERT INTO EMPLOYEE2 values (105,'RAM','ANDROID DEVLOPER');

DECLARE
   v_empid EMPLOYEE2.empId%TYPE := 101;
   v_empname EMPLOYEE2.empName%TYPE;
   v_designation EMPLOYEE2.designation%TYPE;

BEGIN
   SELECT empName, designation
   INTO v_empname, v_designation
   FROM EMPLOYEE2
   WHERE empId = v_empid;

   DBMS_OUTPUT.PUT_LINE('ID: ' || v_empid);
   DBMS_OUTPUT.PUT_LINE('Name: ' || v_empname);
   DBMS_OUTPUT.PUT_LINE('Designation: ' || v_designation);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee not found');
   WHEN TOO_MANY_ROWS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('More than one employee found');
END;
```

OUTPUT:

Results Explain Describe Saved SQL History

```
ID: 101
Name: xyz
Designation: ANDROID DEVLOPER

Statement processed.
```

0.00 seconds

Q21. Write a PL/SQL code to update the salary of employees who earn less than the average salary using cursor.

```
BEGIN
   UPDATE EMPLOYEE2
   SET salary = salary + (salary * 0.10)
   WHERE salary < (SELECT AVG(salary) FROM EMPLOYEE2);

   COMMIT;
END;
/
SELECT * FROM EMPLOYEE2;
```

OUTPUT :

Results Explain Describe Saved SQL History

EMPID	EMPNAME	DESIGNATION	SALARY
105	RAM	ANDROID DEVLOPER	27500
104	RAJ	ML DEVLOPER	22000
103	SHAM	DEVOPS DEVLOPER	50000
102	wyz	Frontend DEVLOPER	40000
101	xyz	ANDROID DEVLOPER	33000

rows returned in 0.00 seconds CSV Export

Q22 Write a row trigger to insert the existing values of the salary table into a new table when the salary table is updated.

```
CREATE TABLE SALARY_BACKUP (
   empId NUMBER,
   old_salary NUMBER,
   updated_on DATE
);

CREATE OR REPLACE TRIGGER trg_salary_update
BEFORE UPDATE ON EMPLOYEE2
FOR EACH ROW
BEGIN
   INSERT INTO SALARY_BACKUP
   VALUES (:OLD.empId, :OLD.salary, SYSDATE);
END;
/

UPDATE EMPLOYEE2
SET salary = salary + 1000
WHERE empId = 101;

COMMIT;
SELECT * FROM SALARY_BACKUP;
```

OUTPUT:

EMPID	OLD_SALARY	UPDATED_ON
101	33000	31-MAR-26

1 rows returned in 0.01 seconds CSV Export

Q23. Write a PL/SQL procedure to find the number of students ranging from 100–70%, 69–60%, 59–50% & below 49% in each course from the STUDENT_COURSE table. The counts should be passed as parameters.

```
CREATE TABLE STUDENT_COURSE (
   id NUMBER,
   percentage NUMBER
);

INSERT INTO STUDENT_COURSE VALUES (1, 85);
INSERT INTO STUDENT_COURSE VALUES (2, 72);
INSERT INTO STUDENT_COURSE VALUES (3, 65);
INSERT INTO STUDENT_COURSE VALUES (4, 55);
INSERT INTO STUDENT_COURSE VALUES (5, 45);
INSERT INTO STUDENT_COURSE VALUES (6, 30);
```

```
CREATE OR REPLACE PROCEDURE student_count_proc(
   a OUT NUMBER,
   b OUT NUMBER,
   c OUT NUMBER,
   d OUT NUMBER
)
```

```
AS
BEGIN
   SELECT COUNT(*) INTO a FROM STUDENT_COURSE WHERE percentage >= 70;
   SELECT COUNT(*) INTO b FROM STUDENT_COURSE WHERE percentage >= 60 AND percentage <
70;
   SELECT COUNT(*) INTO c FROM STUDENT_COURSE WHERE percentage >= 50 AND percentage <
60;
   SELECT COUNT(*) INTO d FROM STUDENT_COURSE WHERE percentage < 50;
END;
/
```

```
DECLARE
   x NUMBER;
   y NUMBER;
   z NUMBER;
   w NUMBER;
BEGIN
   student_count_proc(x, y, z, w);

   DBMS_OUTPUT.PUT_LINE('70-100: ' || x);
   DBMS_OUTPUT.PUT_LINE('60-69: ' || y);
   DBMS_OUTPUT.PUT_LINE('50-59: ' || z);
   DBMS_OUTPUT.PUT_LINE('Below 50: ' || w);
END;
/
```

OUTPUT:

Results Explain Describe Saved SQL History

```
70-100: 2
60-69: 1
50-59: 1
Below 50: 2

Statement processed.
```

0.02 seconds

Q24Create a store function that accepts 2 numbers and returns the addition of passed values. Also, write the code to call your function.

```
CREATE OR REPLACE FUNCTION add_two_numbers(a NUMBER, b NUMBER)
RETURN NUMBER
IS
BEGIN
   RETURN a + b;
END;
/
DECLARE
   result NUMBER;
BEGIN
   result := add_two_numbers(10, 20);
   DBMS_OUTPUT.PUT_LINE('Sum = ' || result);
END;
/
```

OUTPUT:

```
Sum = 30

Statement processed.
```

0.00 seconds

Q25. Write a PL/SQL function that accepts the department number and returns the total salary of the department. Also write a function call.

```
-- Create new table
CREATE TABLE EMP_DEPT (
   empId NUMBER,
   empName VARCHAR2(40),
   salary NUMBER,
   department_id NUMBER
);

-- Insert sample data
INSERT INTO EMP_DEPT VALUES (101, 'A', 30000, 1);
INSERT INTO EMP_DEPT VALUES (102, 'B', 40000, 1);
INSERT INTO EMP_DEPT VALUES (103, 'C', 50000, 2);
INSERT INTO EMP_DEPT VALUES (104, 'D', 20000, 2);
INSERT INTO EMP_DEPT VALUES (105, 'E', 25000, 3);
```

```
CREATE OR REPLACE FUNCTION total_salary(dept_id NUMBER)
RETURN NUMBER
IS
   total NUMBER;
BEGIN
   SELECT SUM(salary) INTO total
```

```
    FROM EMP_DEPT
    WHERE department_id = dept_id;

    RETURN total;
END;
/
```

```
SELECT total_salary(1) FROM dual;
```

OUTPUT:

Results Explain Describe Saved SQL History

TOTAL_SALARY(1)
70000

1 rows returned in 0.00 seconds CSV Export

Q26Write a program to illustrate user-defined exceptions, built-in exceptions, and raise application error exceptions.

```
DECLARE
   my_exception EXCEPTION;
   num NUMBER := 0;
   result NUMBER;
BEGIN
   result := 10 / num;
   IF num = 0 THEN
      RAISE my_exception;
   END IF;
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('Built-in Exception: Divide by zero');
   WHEN my_exception THEN
      DBMS_OUTPUT.PUT_LINE('User-defined Exception occurred');
   WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR(-20001, 'Application Error occurred');
END;
/
```

OUTPUT:

Results Explain Describe Saved SQL History

```
Built-in Exception: Divide by zero

Statement processed.
```

0.00 seconds

27 . **Write a program to Reverse a String Using PL/SQL Block.**

```
DECLARE
   str VARCHAR2(50) := 'HELLO';
   rev VARCHAR2(50) := '';
   i NUMBER;
BEGIN
   FOR i IN REVERSE 1..LENGTH(str) LOOP
      rev := rev || SUBSTR(str, i, 1);
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Original: ' || str);
   DBMS_OUTPUT.PUT_LINE('Reversed: ' || rev);
END;
/
```

OUTPUT:

```
Original: HELLO
Reversed: OLLEH

Statement processed.

0.00 seconds
```

28. **Create a trigger that records changes on an EMPLOYEES table (INSERT, UPDATE, DELETE) into an EMPLOYEES_AUDIT table. Include details like EMPLOYEE_ID, OPERATION_TYPE, TIMESTAMP.**

```
CREATE TABLE EMPLOYEES (
   employee_id NUMBER,
   name VARCHAR2(50),
   salary NUMBER
);

CREATE TABLE EMPLOYEES_AUDIT (
   employee_id NUMBER,
   operation_type VARCHAR2(10),
   action_date DATE
);

CREATE OR REPLACE TRIGGER emp_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
BEGIN
   IF INSERTING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:NEW.employee_id, 'INSERT', SYSDATE);

   ELSIF UPDATING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:NEW.employee_id, 'UPDATE', SYSDATE);

   ELSIF DELETING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:OLD.employee_id, 'DELETE', SYSDATE);
   END IF;
```

```
END;
/

INSERT INTO EMPLOYEES VALUES (1, 'A', 30000);

UPDATE EMPLOYEES SET salary = 35000 WHERE employee_id = 1;

DELETE FROM EMPLOYEES WHERE employee_id = 1;
SELECT * FROM EMPLOYEES_AUDIT;
```

OUTPUT:

```
DELETE FROM EMPLOYEES WHERE employee_id = 1;
SELECT * FROM EMPLOYEES_AUDIT;
```

Results Explain Describe Saved SQL History

EMPLOYEE_ID	OPERATION_TYPE	ACTION_DATE
1	INSERT	31-MAR-26
1	UPDATE	31-MAR-26
1	DELETE	31-MAR-26

3 rows returned in 0.00 seconds CSV Export

Q29 **Write a PL/SQL block to demonstrate the use of aggregate functions (COUNT, SUM, AVG, MAX, MIN).**

```
DECLARE
   total NUMBER;
   avg_sal NUMBER;
   max_sal NUMBER;
   min_sal NUMBER;
   count_emp NUMBER;
BEGIN
   SELECT
      COUNT(*),
      SUM(salary),
      AVG(salary),
      MAX(salary),
      MIN(salary)
   INTO
      count_emp,
      total,
      avg_sal,
      max_sal,
      min_sal
   FROM EMP_DEPT;

   DBMS_OUTPUT.PUT_LINE('Count = ' || count_emp);
   DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
   DBMS_OUTPUT.PUT_LINE('Average = ' || avg_sal);
   DBMS_OUTPUT.PUT_LINE('Max = ' || max_sal);
   DBMS_OUTPUT.PUT_LINE('Min = ' || min_sal);
END;
```

/

OUTPUT:

```
DBMS_OUTPUT.PUT_LINE('Average = ' || avg_sal);
```

Results Explain Describe Saved SQL History

```
Count = 5
Sum = 165000
Average = 33000
Max = 50000
Min = 20000

Statement processed.
```

0.02 seconds

Q30. **Write a PL/SQL program using a cursor to display employee details.**

```
DECLARE
   CURSOR emp_cursor IS
      SELECT empId, empName, salary FROM EMP_DEPT;

   v_id EMP_DEPT.empId%TYPE;
   v_name EMP_DEPT.empName%TYPE;
   v_salary EMP_DEPT.salary%TYPE;

BEGIN
   OPEN emp_cursor;

   LOOP
      FETCH emp_cursor INTO v_id, v_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_salary);
   END LOOP;

   CLOSE emp_cursor;
END;
/
```

OUTPUT:

Results Explain Describe Saved SQL History

```
101 A 30000
102 B 40000
103 C 50000
104 D 20000
105 E 25000

Statement processed.
```

0.00 seconds